

MATH 458: Scientific Computing II

Exam Number: 20892768

August 24, 2026

1 Purpose

This report covers the bonus component of the MATH 458 project: MPI-parallel versions of matrix-vector and matrix-matrix multiplication, alongside the vector-vector multiplication already parallelised in Part II. All three operations are implemented in a single distributed-memory program, timed at a fixed problem size $N = 2000$ while varying the number of MPI processes P from 1 to 8, with results written to `20892768_bonus.txt`.

2 Implementation

2.1 Reused serial kernels

The underlying arithmetic in each operation is unchanged from Part I: the same `vector_vector`, `matrix_vector`, and `matrix_matrix` functions, with their original prototypes and row and major flat indexing, are called directly by every MPI rank on that rank's own local data. Parallelism is added entirely in `main.c`, through data distribution and result collection; the mathematical kernels themselves were not modified.

2.2 Data distribution strategy

Vector-vector multiplication reuses the design from Part II: the root rank (rank 0) generates the full vectors, splits them into contiguous blocks with `MPI_Scatterv` (allowing for uneven splits when N is not divisible by P), and combines each rank's partial dot product with `MPI_Reduce` using `MPI_SUM`.

Matrix-vector and matrix-matrix multiplication use a row-wise decomposition of matrix A. The root rank generates the full $N \times N$ matrix A (and matrix B, for matrix-matrix), together with vector A. A contiguous block of *rows* of A is sent to each rank with `MPI_Scatterv`, using a row-count array that assigns one extra row to the first $N \bmod P$ ranks so that every row is distributed even when N does not divide evenly by P . Because each output row of matrix-vector or matrix-matrix multiplication depends on the *entire* vector or matrix B, vector A and matrix B are broadcast in full to every rank with `MPI_Bcast`, rather than split. Each rank then calls the existing serial `matrix_vector` or `matrix_matrix` function on its own row-block, and the results are collected back onto the root rank with `MPI_Gatherv`, the inverse operation of `MPI_Scatterv`.

2.3 Correctness verification

Before the timed $N = 2000$ run, the full pipeline was verified at a small size ($N = 8$, $P = 4$) by comparing the MPI program's output against hand-computed values: the dot product, the first

element of the matrix-vector result, and the first element of the matrix-matrix result all matched exact hand calculations (168, 140, and 91 respectively), confirming that the scatter/broadcast/gather logic distributes and reassembles data correctly before it was trusted at the full problem size.

2.4 Timing method

Each of the three operations is timed separately using `MPI_Wtime()`, which measures per-process CPU time and is less suitable once execution is spread across processes). Each timed region covers the relevant `MPI_Scatterv/MPI_Bcast` call, the local computation, and the corresponding `MPI_Reduce/MPI_Gatherv` call, so that communication overhead is included in the reported time rather than only the arithmetic.

3 Experimental Setup

All runs were performed on the same development machine used for Parts I and II (approximately 3.2 GB RAM), which has fewer than 8 physical cores. Runs with $P >$ (physical core count) were executed using Open MPI's `-oversubscribe` flag, which permits more MPI processes than the machine reports as available slots. This is noted explicitly because it affects the interpretation of the $P = 4$ to 8 results in Section 5: beyond the true core count, additional ranks compete for the same physical hardware rather than obtaining genuine additional parallelism.

The problem size was fixed at $N = 2000$ for all process counts, as specified for the bonus component. At this size, the two $N \times N$ double matrices held on the root rank require approximately 64 MB in total, well within the available memory on the development machine.

4 Results

Table 1: Measured MPI wall times in seconds at $N = 2000$, exam number 20892768.

P	V.V time (s)	M.V time (s)	M.M time (s)	M.M speedup vs. $P=1$
1	0.000056	0.056356	140.182203	1.00×
2	0.000111	0.053728	69.105394	2.03×
3	0.000120	0.057367	92.241776	1.52×
4	0.086857	0.061399	80.578626	1.74×
5	0.151030	0.070239	79.142854	1.77×
6	0.188279	0.069459	80.254853	1.75×
7	0.304134	0.064514	82.721943	1.69×
8	0.004378	0.110561	82.400637	1.70×

5 Discussion

5.1 Matrix-matrix multiplication

Matrix-matrix multiplication shows the clearest and most expected result: wall time drops from 140.18 s at $P = 1$ to 69.11 s at $P = 2$, a speedup of 2.03×, close to the ideal value of 2× for doubling the process count. Beyond $P = 2$, however, time does not continue to fall; it rises to 92.24 s at

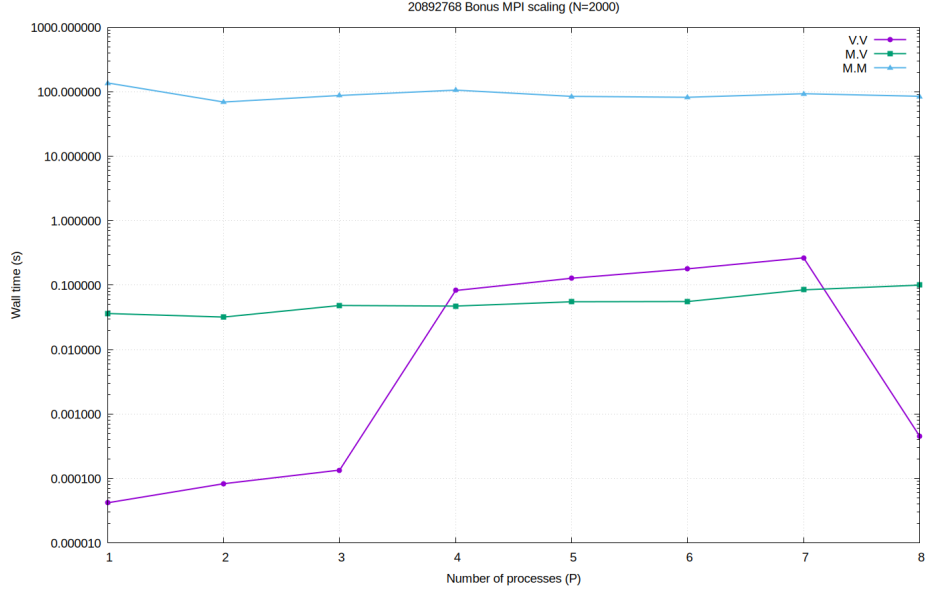


Figure 1: V.V, M.V, and M.M wall time versus process count P at $N = 2000$, exam number 20892768 (logarithmic vertical axis).

$P = 3$ and then settles in the 79-83s range for $P = 4$ through $P = 8$, giving speedups of only $1.5\text{-}1.8\times$ relative to the serial run despite using up to eight processes. This flattening, rather than continued improvement, is consistent with the machine having fewer than eight physical cores: once the process count exceeds the true core count, additional ranks are scheduled onto cores already in use rather than genuinely running in parallel, and the extra `MPI_Scatterv`/`MPI_Gatherv` communication for more, smaller row-blocks adds overhead without a matching computational benefit.

5.2 Matrix-vector multiplication

Matrix-vector times stay small and relatively flat across all P (roughly 0.05-0.11 s), with no clear speedup trend. This is expected: matrix-vector multiplication is an $O(N^2)$ operation, and at $N = 2000$ the total arithmetic work is small enough that it completes in well under a tenth of a second even serially, so the fixed cost of broadcasting the full vector and matrix B , and gathering the row results, is comparable to or larger than the computation itself. The operation is too cheap at this N for the row-decomposition strategy to show a clear benefit.

5.3 Vector-vector multiplication

Vector-vector multiplication shows the opposite of a speedup: wall time generally increases with P , from 0.000056 s at $P = 1$ up to 0.30 s at $P = 7$ (the sharp drop to 0.0044 s at $P = 8$ is treated as measurement noise given how small and communication-dominated this operation is). This matches the point made explicitly in the course material that speedup is limited by the serial fraction of the code *plus communication* (Amdahl's law). The vector-vector dot product at $N = 2000$ is an almost trivial amount of arithmetic; the cost of the `MPI_Scatterv` and `MPI_Reduce` calls needed to split and recombine that tiny amount of work is essentially fixed regardless of how little computation each rank receives, so adding more processes only adds communication overhead without any offsetting

computational benefit. This is a genuine, and instructive, illustration of a case where parallelisation is inappropriate given the problem size.

6 Conclusion

The bonus component successfully parallelises matrix-vector and matrix-matrix multiplication using MPI, reusing the unmodified serial kernels from Part I with a row-wise decomposition of matrix A, full broadcast of the shared vector/matrix operand, and gathered results, following the same root-generates-and-distributes design already used for vector-vector multiplication in Part II. The results show a clear, expected speedup for the computationally heavy $O(N^3)$ matrix-matrix operation up to the machine's physical core count, followed by a plateau consistent with oversubscription beyond that point, while the comparatively cheap matrix-vector and vector-vector operations at $N = 2000$ show that communication overhead can dominate, or even reverse, the benefit of parallelisation when the computational work per process is too small. This is consistent with Amdahl's law and directly demonstrates why the choice of problem size matters when deciding whether an operation is worth parallelising.